INSPIRE AI

# STAND-UP

## Standup Meeting



© 2026 Inspire | Confidential

# Share Yesterday with ORID

| | |
|---|---|
| **Objective** | What happened? Give one concrete event. |
| **Reflective** | How did you feel? Choose what to share. |
| **Interpretive** | What did you learn from it? |
| **Decisional** | What will you do next? |

15 minutes for the whole standup. Use the actual group size to give everyone a turn.

© 2026 Inspire | Confidential

INSPIRE AI

# REVIEW

## Share Graded TDD Work



© 2026 Inspire | Confidential

# Share Your Graded TDD Work

| | |
|---|---|
| **Your submitted change** | Show the member shipping threshold and the migrated assertions. |
| **Your code and commits** | Use submitted tests and Git history to explain the TDD steps. No saved output or screenshots are required. |
| **Your design choice** | Explain why you continued or stopped using Simple Design. |

15 minutes. Equal sharing and feedback time for every member. Keep the submitted version and score.

© 2026 Inspire | Confidential

# Why Understanding Still Matters

| | |
|---|---|
| **Code or an explanation** | A useful starting point for investigation. |
| **Engineering judgment** | Predict a result. Explain why. Diagnose a failure. |

Completing a task and understanding a system are different outcomes.

AI can help you read. Check its claims against the target code and evidence.

© 2026 Inspire | Confidential

# Set a Reading Goal

Member order. Same original price, different shipping rule.

| Rule | Original | Member price | Promotion | Goods | Shipping | Total |
|---|---|---|---|---|---|---|
| **Pre-homework rule** | 125.00 | 100.00 | 20.00 | 80.00 | 10.00 | 90.00 |
| **Member-threshold request** | 125.00 | 100.00 | 20.00 | 80.00 | 0.00 | 80.00 |

US-02-E07, US-05-E05. Expected yuan amounts. Observation: Not recorded.

© 2026 Inspire | Confidential

# Build a Checkable Explanation

| | |
|---|---|
| **Reading goal** | Name the behavior or mechanism you need to understand. |
| **Source and explanation** | Locate relevant code. State an explanation you can check. |
| **Check and refine** | Observe execution or read assertions. Revise the explanation if needed. |

Course adaptation of Tie Code and Questions, Step Through the Execution and Write Tests to Understand.

© 2026 Inspire | Confidential

# Follow Calls and Values

| At the call | Inside the method | Back at the caller |
|---|---|---|
| **Argument** | Which parameter receives it? | Which returned value is used? |
| **Condition** | Why does this path run? | How does the result affect the next step? |

Which value? Where did it come from?

A method name or call chain alone cannot explain an amount.

© 2026 Inspire | Confidential

# Track the Object That Changes

| Object | Before the action | After the action |
| --- | --- | --- |
| **O** | The state you read | The state this action changes |
| **P** | A separate object | Check whether its state stays the same |

Keep object identity when you explain state changes.

A returned value and a change to an existing object need separate checks.

© 2026 Inspire | Confidential

# Use One Scenario to Observe Execution

1. Choose the input, initial state and action.
2. Observe the calls, conditions and objects that answer your question.
3. Compare the observation with your explanation.

This run covers this scenario. The complete debugger demonstration follows in the parking project.

© 2026 Inspire | Confidential

# Match Each Claim to Its Evidence

| Evidence | Supports | Limit |
| --- | --- | --- |
| **Source inspection** | A reasoned account of a possible path | It does not record a run |
| **Runtime observation** | Values and actions in this run | It does not cover every scenario |
| **Test assertions** | The results actually checked | They do not observe every internal step |

Open the assertions before saying what a green result proves.

© 2026 Inspire | Confidential

# Use Tests to Check Your Understanding

| | |
|---|---|
| **Read existing assertions** | Reuse a check that already answers the behavior question. |
| **Add a focused check** | Use it when a real evidence gap needs a test. |
| **Compare with the rule** | A result can describe current behavior. Requirements decide what should happen. |

Protection for correct existing behavior can start Green. A compile failure does not prove missing behavior.

© 2026 Inspire | Confidential

# Check an AI Explanation

Hypothetical AI answer, written for teaching

> “This pre-homework version already gives members free shipping at goods 80. All tests are green, so every member-threshold behavior is correct.”

| | |
|---|---|
| **Rule?** | What does the confirmed request say? |
| **Implementation?** | What does this version actually do? |
| **Tests?** | Which results and version do the assertions check? |

© 2026 Inspire | Confidential

# Read the Familiar Calculation

**QuoteCalculator.java** java • demo/tiered-discount • lines 4–9

```
4       public Quote calculate(BigDecimal original, boolean member) {
5           if (original.signum() == 0) {
6               BigDecimal zero = Money.amount("0.00");
7               return new Quote(original, zero, zero, zero, zero, zero);
8           }
9           BigDecimal memberPrice = Money.memberPrice(original, member);
```

src/main/java/training/quote/QuoteCalculator.java

Pre-homework instructor version. calculate receives the original price and membership flag.

© 2026 Inspire | Confidential

# Follow the Goods Value into Shipping

**QuoteCalculator.java** java • demo/tiered-discount • lines 16–19

```
16          BigDecimal goods = memberPrice.subtract(promotion);
17          BigDecimal shipping = shippingFor(goods);
18          BigDecimal total = goods.add(shipping);
19          return new Quote(original, memberPrice, promotion, goods, shipping,
    total);
```

src/main/java/training/quote/QuoteCalculator.java

Pre-homework instructor version. The shipping call receives goods after the existing reductions.

© 2026 Inspire | Confidential

# Read the Existing Shipping Condition

**QuoteCalculator.java** java • demo/tiered-discount • lines 22–28

```
22      private BigDecimal shippingFor(BigDecimal goods) {
23          BigDecimal shippingBasis = goods;
24          BigDecimal shippingThreshold = Money.amount("100.00");
25          boolean freeShipping = shippingBasis.compareTo(shippingThreshold) >= 0;
26          BigDecimal shippingFeeWhenCharged = Money.amount("10.00");
27          return freeShipping ? Money.amount("0.00") : shippingFeeWhenCharged;
28      }
```

src/main/java/training/quote/QuoteCalculator.java

Read the argument, threshold and returned fee. This method receives goods alone.

© 2026 Inspire | Confidential

# Read the Matching Test Call

**Day2CompleteReferenceTest.java** java • demo/tiered-discount • lines 136–138

```
136     @Test void day2Example11() {
137         var quote = new QuoteCalculator().calculate(Money.amount("125.00"),
    true);
138         assertAll(
```

src/test/java/training/quote/Day2CompleteReferenceTest.java

Pre-homework instructor version. The same test also checks the original amount and printed output.

© 2026 Inspire | Confidential

# Check the Existing Amount Assertions

**Day2CompleteReferenceTest.java** java • demo/tiered-discount • lines 140–144

```
140             () -> assertEquals(Money.amount("100.00"), quote.memberPrice(),
    "day2Example11: member_price"),
141             () -> assertEquals(Money.amount("20.00"), quote.promotion(),
    "day2Example11: promotion"),
142             () -> assertEquals(Money.amount("80.00"), quote.goods(),
    "day2Example11: goods"),
143             () -> assertEquals(Money.amount("10.00"), quote.shipping(),
    "day2Example11: shipping"),
144             () -> assertEquals(Money.amount("90.00"), quote.total(),
    "day2Example11: total"),
```

src/test/java/training/quote/Day2CompleteReferenceTest.java

Saved instructor baseline: 34 tests, 0 failures, 0 errors, 0 skipped.
Recorded 2026-10-21. Pre-homework version. Current classroom run: not recorded.

© 2026 Inspire | Confidential

YOUR TURN

# What Does This Version Do, and Why?

**Give a bounded explanation of this version.**

Connect the input, goods, shipping condition and assertions.

**Which claim has the evidence settled?**

**Which other scenarios remain unchecked?**

© 2026 Inspire | Confidential

# Apply the Method to a New Behavior

| Next task | Who leads? | What is checked? |
|---|---|---|
| **Parking demonstration** | Instructor | Explain a complete trace with code, state and assertions |
| **Return trace, with an entry hint** | Learner | Explain which object changes and why |
| **Report trace, without an entry hint** | Learner | Find the path and support the explanation |

Keep the reading method. Entry hints become smaller as you practice.

© 2026 Inspire | Confidential

# BREAK

## Take a Break

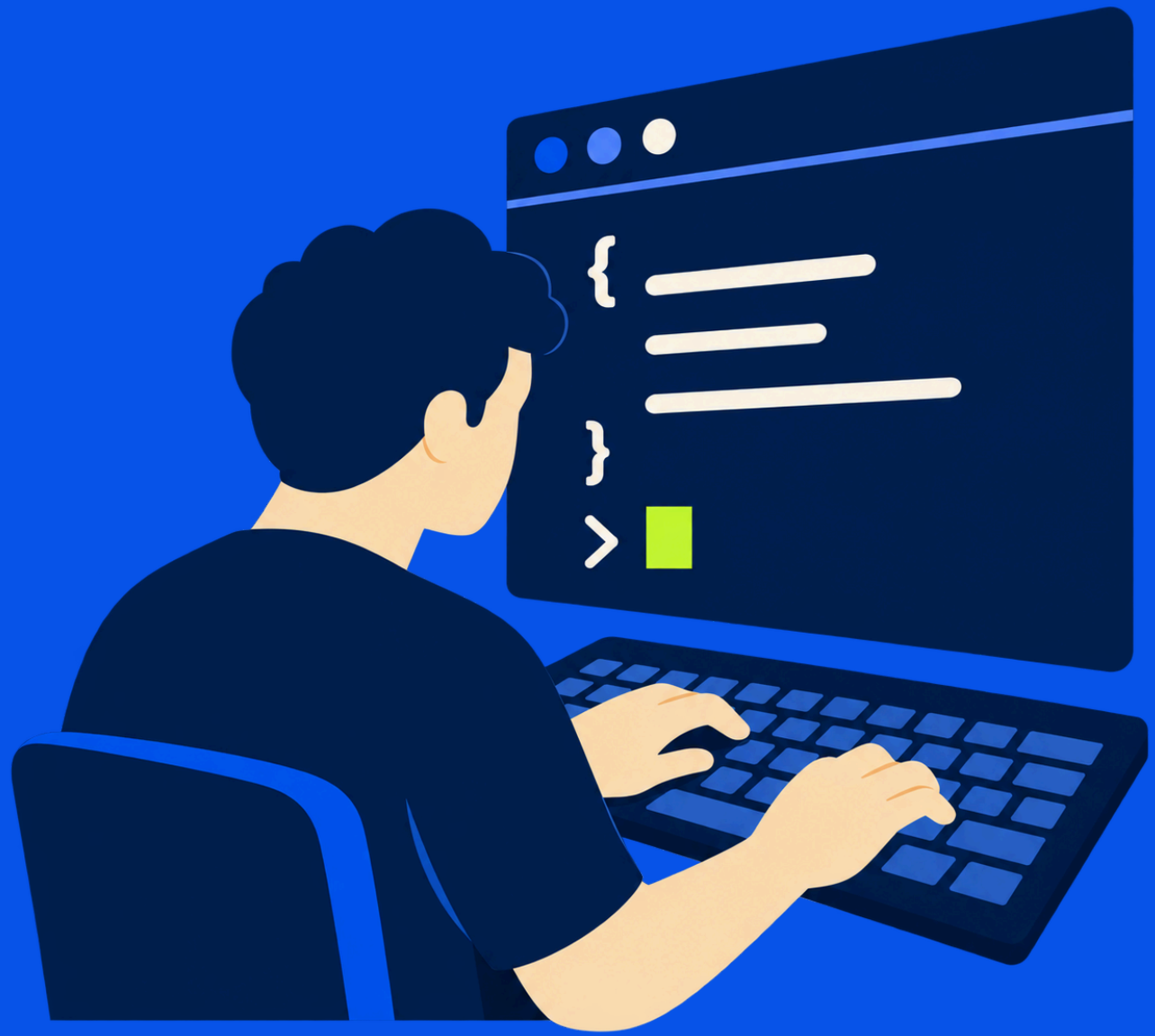


© 2026 Inspire | Confidential

INSPIRE AI

# DEMO

## Trace a Behavior in Unfamiliar Code



© 2026 Inspire | Confidential

# Assign the Next Car by Registration Order

> As parking attendant, I want each car to enter the first registered lot that still has space, so that spaces are assigned in the agreed order.

Original request • US-01

© 2026 Inspire | Confidential

# The Parking Rules Set the Expected Result

| | |
|---|---|
| **US-01-AC01** | Choose the first lot in registration order that still has a space. |
| **US-01-AC02** | After a successful parking action, reduce that lot's remaining spaces by one and issue a ticket for returning the car. |

Selected criteria for US-01 • rules, not recorded test results

The project already contains production code and runnable tests.

© 2026 Inspire | Confidential

# Read the Known Result Before the Code

| | | |
|---|---|---|
| **Given** | Registration order<br>**A, B** | Remaining spaces<br>**A: 1, B: 3** |
| **When** | Park one car. | |
| **Then**<br>Expected result | Assigned lot<br>**A** | Remaining spaces<br>**A: 0, B: 3** |

Observation: Not recorded

Source example: US-01-E01

Explain how this version produces the given result. The result is already specified.

© 2026 Inspire | Confidential

INSPIRE AI

# PRACTICE

## Trace Two Related Behaviors



© 2026 Inspire | Confidential

# Use a Valid Ticket to Return the Car

| | |
|---|---|
| **US-01-AC03** | Returning a car with a valid ticket restores one space in the corresponding lot. |

Selected criteria for US-01 • rules, not recorded test results

Continue in your existing parking project.

© 2026 Inspire | Confidential

# The Return Has a Known State Change

| | | |
|---|---|---|
| **Given** | Ticket for lot<br>**A** | Remaining spaces<br>**A: 0, B: 3** |
| **When** | Return the car using this ticket. | |
| **Then**<br>Expected result | Remaining spaces<br>**A: 1, B: 3** | |

Observation: Not recorded

Source example: US-01-E02

© 2026 Inspire | Confidential

# Trace the Return with One Entry Hint

1. Start at ParkingService.returnCar.
2. Find the matching existing test.
3. Identify the object that the ticket selects and the object actually updated.
4. Run or debug the test and save a short explanation with its evidence.

Save your result before the break. Record any help.

© 2026 Inspire | Confidential

# BREAK

## Take a Break



© 2026 Inspire | Confidential

# Read the Report Requirement

> As parking attendant, I want to see the remaining spaces in each lot, so that I can inspect availability without changing the next parking allocation.

Original request • US-02

© 2026 Inspire | Confidential

# The Report Orders the Remaining Spaces

| | |
|---|---|
| **US-02-AC01** | Show each lot's remaining spaces in descending order. Lots with different remaining spaces must be ordered correctly. |

Selected criteria for US-02 • rules, not recorded test results

© 2026 Inspire | Confidential

# Find the Code Behind This Report Result

| | | |
|---|---|---|
| **Given** | Registration order<br>**A, B** | Remaining spaces<br>**A: 1, B: 3** |
| **When** | View the availability report. | |
| **Then**<br>Expected result | Report order<br>**B, A** | |

Observation: Not recorded

Source example: US-02-E01

Find the entry and the relevant test yourself. Keep the confirmed example available.

© 2026 Inspire | Confidential

YOUR TURN

# Support Your Report Explanation

**Save source locations, observations and a supported conclusion. Ask for a bounded hint when blocked.**

Use the same reading guide in your own project.

**Where is the behavior produced?**

**Which values explain the order?**

**What do the assertions check?**

© 2026 Inspire | Confidential

INSPIRE AI

# PRACTICE

## Plan a Small Change from Its Effects



© 2026 Inspire | Confidential

# See All Lots, Including Full Ones

> As a parking attendant, I want the report to show full lots and their status, so that I can see all parking lots.

Original request • US-03

© 2026 Inspire | Confidential

# Keep the Change Scope Explicit

| | |
|---|---|
| **US-03-AC01** | Include full lots with zero remaining spaces and status FULL. Other lots must not display FULL. |

Selected criteria for US-03 • rules, not recorded test results

Replace hiding full lots. Keep report ordering and parking allocation unchanged. Equal-space order is still unspecified.

© 2026 Inspire | Confidential

# The Current Report Hides a Full Lot

| | | |
|---|---|---|
| **Given** | Registration order<br>**A, B** | Remaining spaces<br>**A: 0, B: 3** |
| **When** | View the current availability report. | |
| **Then**<br>Expected result | Report rows<br>**Lot: B, Remaining spaces: 3, Status: AVAILABLE** | |

Observation: Not recorded

Source example: US-02-E03

© 2026 Inspire | Confidential

# The Request Adds a FULL Row

| | | |
|---|---|---|
| **Given** | Registration order<br>**A, B** | Remaining spaces<br>**A: 0, B: 3** |
| **When** | View the report after the proposed change. | |
| **Then**<br>Expected result | Report rows<br>**Lot: B, Remaining spaces: 3, Status: AVAILABLE, Lot: A, Remaining spaces: 0, Status: FULL** | |

Observation: Not recorded

Source example: US-03-E01

This is the proposed result. Today you plan its effects and keep it unimplemented.

© 2026 Inspire | Confidential

YOUR TURN

# Find Another Supported Effect

**Locate evidence before asking for a hint. Add investigation tasks only for real unknowns. Keep FULL unimplemented.**

Continue the impact task list in README.

**Which other work is affected?**

**Which existing behavior should stay?**

**What check would show completion?**

© 2026 Inspire | Confidential

# BREAK

## Lunch Break



© 2026 Inspire | Confidential

# MINI-GAME

## Optional Mini-game



© 2026 Inspire | Confidential

YOUR TURN

# Which Classes Come to Mind?

**Name candidates, then connect each to a responsibility.**

Use the parking behavior and code you have already read.

**Who chooses a lot for the next car?**

**Who decides the report order?**

© 2026 Inspire | Confidential

# Assign Knowledge, Decisions and Guarantees

**ParkingService.java** java • main • lines 43–44

```
43              int spaces = remaining(lot);
44              if (spaces > 0) report.add(new ReportRow(lot.id(), spaces,
    "AVAILABLE"));
```

src/main/java/training/parking/ParkingService.java

Knows what? Decides what? Does what? Guarantees what?

© 2026 Inspire | Confidential

# Collaborate Through Requests and Results

| | |
|---|---|
| **ParkingService** | Provides a ReportRow to add. It chooses the row values. |
| **AvailabilityReport** | Collects rows and provides them through rows(). |
| **The caller** | Relies on the returned row list and its stated behavior. |

What must the caller provide? What result or guarantee can it rely on?

© 2026 Inspire | Confidential

# Encapsulate Rules and Representation

**AvailabilityReport.java** java • main • lines 6–10

```
6  public class AvailabilityReport {
7      private final List<ReportRow> rows = new ArrayList<>();
8
9      public void add(ReportRow row) { rows.add(row); }
10     public List<ReportRow> rows() { return List.copyOf(rows); }
```

src/main/java/training/parking/AvailabilityReport.java

Private storage and a list snapshot protect representation. A query can be a useful interface.

© 2026 Inspire | Confidential

# Compare Cohesion, Coupling and Cost

**ParkingService.java** java • main • lines 39–40

```
39          List<ParkingLot> reportOrder = new ArrayList<>(registeredLots);
40          reportOrder.sort(Comparator.comparingInt(this::remaining).reversed());
```

src/main/java/training/parking/ParkingService.java

| | |
|---|---|
| **Cohesion** | Do the related responsibilities support one purpose? |
| **Coupling and cost** | What must callers know? What extra request or call order would a move add? |

© 2026 Inspire | Confidential

YOUR TURN

# Where Could the Ordering Decision Belong?

**3 minutes for individual judgment. Compare the current placement with another reasonable one.**

Open the current report path. First identify the data ordering needs.

**What would the caller stop knowing?**

**What new dependency or step would it need?**

© 2026 Inspire | Confidential

YOUR TURN

# What Boundary Did You Choose, and Why?

**About 45 seconds for each explanation. Use the rest of this three-minute segment for feedback.**

Hear two actual judgments before the instructor comparison.

**What does the caller stop knowing?**

**What dependency or call order does the choice add?**

© 2026 Inspire | Confidential

# Explain the Boundary and Its Cost

| Candidate | Reason to consider it | Cost or limitation |
| --- | --- | --- |
| **Keep ordering in ParkingService** | Reuse the current flow and interface | The service keeps the report ordering rule |
| **Let AvailabilityReport order its rows** | Keep ordering with the row collection | Add a request and required call order |

First hear two actual judgments. Then compare these instructor candidates. 3 minutes including feedback.

© 2026 Inspire | Confidential

# Carry the Choice into a Tested Refactor

1. State the choice, its reason and its cost.
2. Make a small code change.
3. Check public interfaces and business behavior separately.
4. Use the diff to explain the responsibility and dependency change.

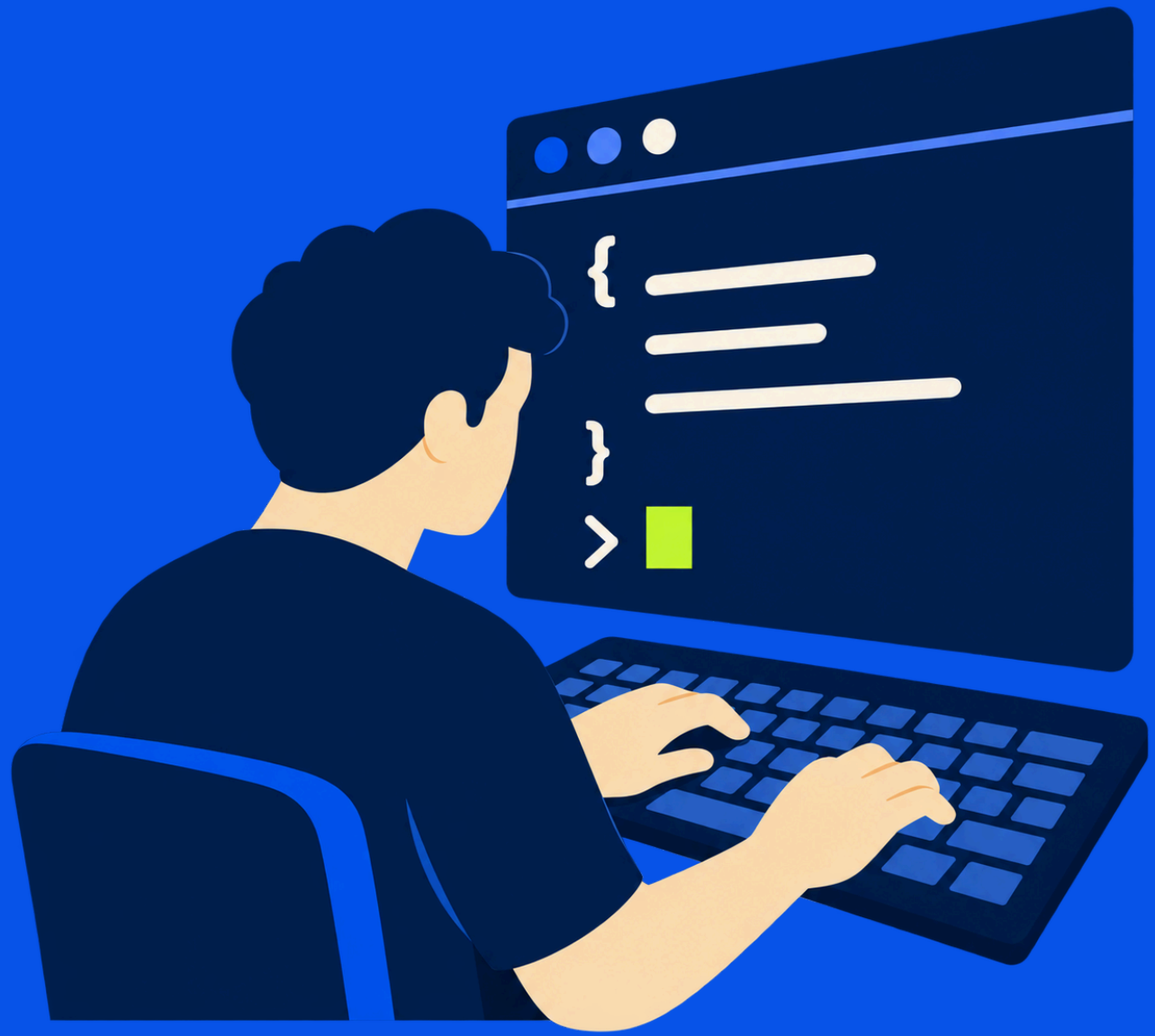
Demo: report ordering. Practice: another responsibility. Existing list-copy protection is already present.

© 2026 Inspire | Confidential

INSPIRE AI

# DEMO

## Choose Where One Behavior Belongs



© 2026 Inspire | Confidential

# Connect the Choice to the Sorting Objects

| Current flow | Chosen demonstration |
| --- | --- |
| **Sort a copy of ParkingLot objects** | Collect ReportRow objects, then request their ordering |
| **Service knows the descending comparison** | Report owns the row-ordering rule |

8 minutes to reconnect the choice and check protection. Keep current row values, filtering and registration order.

© 2026 Inspire | Confidential

# BREAK

## Take a Break



© 2026 Inspire | Confidential

INSPIRE AI

# PRACTICE

## Refactor an Object Responsibility with Tests



© 2026 Inspire | Confidential

# Refactor a Remaining-Space Responsibility

**Assign a responsibility to an object, implement the change and explain how collaboration or encapsulation improves.**

1. Locate remaining-space knowledge and compare two owners.
2. Protect the behavior, move one responsibility and switch the callers.
3. Check results and explain the benefit, alternative and cost.

**Produce**

refactored learner code and brief responsibility and collaboration reasons

actual behavior checks and the remaining task or stopping reason in the same repository

30 minutes • whole activity

Source activity: d04-s08

Compare owners. Protect behavior, switch callers, then test, inspect and commit. Explain the boundary and its cost. Keep FULL unimplemented.

© 2026 Inspire | Confidential

# BREAK

## Take a Break



© 2026 Inspire | Confidential

# PRACTICE

## Refactor a Complete Flow into Collaborating Objects



© 2026 Inspire | Confidential

# Transfer the Same Skills to Stock

**Apply tracing, protection and small-step refactoring to a complete stock flow using a common responsibility contract.**

1. Trace stock checking and reservation in the working starter.
2. Protect the selected risks, then move shared aggregation into an immutable request used by both entries.
3. Keep a shortage-detail impact plan in README. Save actual results, help and remaining inventory work.

## Produce

saved stock project with protection tests, actual responsibility refactor and a brief README explanation

impact and refactoring task list in the same README, with feedback and remaining work

100 minutes • whole activity

Source activity: d04-s09

© 2026 Inspire | Confidential

# Check a Request Without Changing Stock

> As a warehouse operator, I want to check whether a request can be met without changing actual stock.

Original request • US-01

© 2026 Inspire | Confidential

# Understand the Read-Only Checking Rules

| | |
|---|---|
| **US-01-AC01** | Add quantities for the same item before comparing the total with available stock. |
| **US-01-AC02** | Return true when every item total is no greater than its available stock. Otherwise return false. |
| **US-01-AC03** | Checking a request must leave all stock values unchanged. |

Selected criteria for US-01 • rules, not recorded test results

© 2026 Inspire | Confidential

# Checking Is Read-Only

| | | |
|---|---|---|
| **Given** | Stock<br>**A: 4, B: 2** | Request in order<br>**A × 1, B × 1, A × 2** |
| **When** | Check the request. | |
| **Then**<br>Expected result | Can fulfil<br>**true** | Stock<br>**A: 4, B: 2** |

Observation: Not recorded

Source example: US-01-E01

© 2026 Inspire | Confidential

# Reserve Everything or Keep All Stock

> As a warehouse operator, I want a request to be reserved in full or all stock to remain unchanged.

Original request • US-02

© 2026 Inspire | Confidential

# Preserve the Existing Reservation Guarantees

| | |
|---|---|
| **US-02-AC01** | A request that can be met returns true and reduces each item stock by its total requested quantity. |
| **US-02-AC02** | A request that cannot be met returns false and leaves all stock unchanged. Changing the order of its lines must not change this outcome. |

Selected criteria for US-02 • rules, not recorded test results

© 2026 Inspire | Confidential

# A Successful Reservation Deducts the Totals

| | | |
|---|---|---|
| **Given** | Stock<br>**A: 5, B: 3** | Request in order<br>**A × 1, B × 2, A × 1** |
| **When** | Reserve the request. | |
| **Then**<br>Expected result | Accepted<br>**true** | Stock<br>**A: 3, B: 1** |

Observation: Not recorded

Source example: US-02-E01

Run the existing baseline before refactoring. Separate setup failures from behavior results.

© 2026 Inspire | Confidential

# A Rejected Reservation Keeps All Stock

| | | |
|---|---|---|
| **Given** | Stock<br>**A: 5, B: 1** | Request in order<br>**A × 2, B × 2** |
| **When** | Reserve the request when the second item has insufficient stock. | |
| **Then**<br>Expected result | Accepted<br>**false** | Stock<br>**A: 5, B: 1** |

Observation: Not recorded

Source example: US-02-E02

The all-or-none guarantee is existing behavior. Read this expected result before locating its implementation.

© 2026 Inspire | Confidential

# Use the Shared Responsibility Contract

| | |
|---|---|
| **Request object** | Combine repeated-item quantities into immutable demand. Both operations use it. |
| **Inventory object** | Own the whole-request decision and the all-or-none update guarantee. |
| **Public services** | Keep their entries. Construct the request and delegate. |

Move request aggregation in class. Complete inventory ownership in Homework. Equivalent names are allowed.

© 2026 Inspire | Confidential

# Protect Exact-Stock Acceptance

| Example and action | Given | Expected result |
| --- | --- | --- |
| **US-01-E02**<br>Check the request whose total for A exactly equals available stock. | Stock<br>**A: 4, B: 2**<br>Request in order<br>**A × 2, B × 1, A × 2** | Can fulfil<br>**true**<br>Stock<br>**A: 4, B: 2** |
| **US-02-E04**<br>Reserve the request whose total for A exactly equals stock. | Stock<br>**A: 4, B: 2**<br>Request in order<br>**A × 2, B × 1, A × 2** | Accepted<br>**true**<br>Stock<br>**A: 0, B: 1** |

Selected expectations. Observation: Not recorded.

© 2026 Inspire | Confidential

# Protect Rejection of a Combined Shortage

| Example and action | Given | Expected result |
| --- | --- | --- |
| **US-01-E03**<br>Check the request whose individual lines fit but whose total for A exceeds stock. | Stock<br>**A: 4, B: 2**<br>Request in order<br>**A × 3, B × 1, A × 2** | Can fulfil<br>**false**<br>Stock<br>**A: 4, B: 2** |
| **US-02-E05**<br>Reserve the request whose individual lines fit but whose total for A exceeds stock. | Stock<br>**A: 4, B: 2**<br>Request in order<br>**A × 3, B × 1, A × 2** | Accepted<br>**false**<br>Stock<br>**A: 4, B: 2** |

Selected expectations. Observation: Not recorded.

© 2026 Inspire | Confidential

# Protect All Stock in Both Line Orders

| Example and action | Given | Expected result |
| --- | --- | --- |
| **US-02-E02**<br>Reserve the request when the second item has insufficient stock. | Stock<br>**A: 5, B: 1**<br>Request in order<br>**A × 2, B × 2** | Accepted<br>**false**<br>Stock<br>**A: 5, B: 1** |
| **US-02-E03**<br>Reset stock and reserve the same request with the insufficient item first. | Stock<br>**A: 5, B: 1**<br>Request in order<br>**B × 2, A × 2** | Accepted<br>**false**<br>Stock<br>**A: 5, B: 1** |

Selected expectations. Observation: Not recorded.

© 2026 Inspire | Confidential

# Explain What Is Missing

> As a warehouse operator, I want an unsuccessful check to explain how many of each item are missing.

Original request • US-03

© 2026 Inspire | Confidential

# Add Details and Preserve the Existing Guarantees

| | |
|---|---|
| **US-03-AC01** | For each item with a shortage, return the total requested quantity minus available stock. With no shortages, return an empty list. List order is not specified. |

Selected criteria for US-03 • rules, not recorded test results

Keep combined quantities, the fulfilment decision and read-only checking. Preserve successful deduction and all stock after rejection.

© 2026 Inspire | Confidential

# Plan the Proposed Shortage Result

| | | | |
|---|---|---|---|
| **Given** | Stock<br>**A: 4, B: 2** | | Request in order<br>**A × 3, B × 4, A × 2** |
| **When** | Check the request using the proposed shortage-detail result. | | |
| **Then**<br>Expected result | Can fulfil<br>**false** | Shortages<br>**A: 1, B: 2** | Stock<br>**A: 4, B: 2** |

Observation: Not recorded

Source example: US-03-E01

Current checking returns false. The proposed result also explains each shortage. Keep this change unimplemented.

© 2026 Inspire | Confidential

# Plan the Change from Code Evidence

1. Compare the current and proposed results.
2. Follow the affected code, callers and test assertions.
3. Write README tasks with a location, action and completion condition.
4. Include checks for preserved behavior and questions supported by evidence.

Work independently first. Record help. Keep shortage details unimplemented and planned checks unchecked.

© 2026 Inspire | Confidential

INSPIRE AI

# BREAK

## Take a Break



© 2026 Inspire | Confidential

# Move Aggregation into One Immutable Request

1. Choose a safe migration order under passing protection.
2. Make the request own repeated-item totals without exposing mutable totals.
3. Switch both checking and reservation to use it.
4. Remove duplicate aggregation from the services.

Resume your saved project after the break. Run tests, inspect the diff and commit each passing step. Both public entries must use the immutable request; inventory ownership continues in Homework.

© 2026 Inspire | Confidential

INSPIRE AI

# SHOWCASE

## Explain a Responsibility Change and Its Tradeoffs



© 2026 Inspire | Confidential

# Explain the Responsibility You Actually Moved

1. Show the saved code, diff and supporting test.
2. Name the responsibility and its new owner.
3. Explain what callers no longer need to know.
4. Compare an alternative and one added dependency cost.

Two actual changes, about five minutes each. Passing tests and a useful responsibility boundary need separate judgment.

© 2026 Inspire | Confidential